

Data Preparation for Business Analytics — Exam Reference Sheet

SESSION 1: R Basics

RStudio Layout

- Bottom-left: Console (command window) | Top-left: Editor/Script | Top-right: Workspace/History | Bottom-right: Files/Plots/Help
- Set working directory: `setwd("path")` or Tools → Set Working Directory
- Install/load packages: `install.packages("tidyverse")` | `library(tidyverse)` | Clear memory: `rm(list=ls())`

Data Types

Type	Description	Example	Check/Convert
numeric	Real numbers (default)	3.14, 42	<code>is.numeric()</code> / <code>as.numeric()</code>
integer	Whole numbers	42L	<code>is.integer()</code> / <code>as.integer()</code>
character	Text strings	"Hello"	<code>is.character()</code> / <code>as.character()</code>
logical	Boolean (case-sensitive)	TRUE / FALSE	<code>is.logical()</code> / <code>as.logical()</code>
complex	Complex numbers	3+4i	<code>is.complex()</code>

Data Structures

- **Vector:** `c(1,2,3)` | `seq(from, to, by)` | `rep(x, n)`
- **Factor:** `factor(x)` | `ordered(x, levels=c(...), labels=c(...))` — categorical variable with hidden numbers
- **Matrix:** `matrix(data, nrow, ncol)` | Naming: `colnames(m)<-c(...)`, `rownames(m)<-c(...)`
- **Data Frame:** `data.frame()` — like matrix but columns can have different types; add col: `df$newcol <- values`
- **List:** `list(a, b, c)` — heterogeneous collection; access: `mylist[[2]]`; remove: `mylist[4]<-NULL`

Indexing / Subsetting

Expression	Meaning
<code>x[n]</code>	nth element of vector
<code>x[m:n]</code>	mth to nth elements
<code>x[c(k,m,n)]</code>	specific elements
<code>x[x>m & x<n]</code>	elements between m and n
<code>mat[i,j]</code>	element at row i, col j
<code>mat[i,]</code>	row i of matrix
<code>mat[,j]</code>	col j of matrix
<code>x[-c(3,5)]</code>	exclude 3rd and 5th

Matrix multiply: `m1 %*% m2` | Inverse: `solve(m)` | Transpose: `t(m)`

SESSION 2: Data Import

Core readr Functions (tidyverse)

Function	File Type	Key Arguments
<code>read_csv("file.csv")</code>	CSV (comma)	<code>col_names</code> , <code>skip</code> , <code>col_types</code>
<code>read_csv2("file.csv")</code>	CSV (semicolon)	<code>col_names</code> , <code>skip</code> , <code>col_types</code>
<code>read_tsv("file.tsv")</code>	TSV (tab)	<code>col_names</code> , <code>skip</code> , <code>col_types</code>
<code>read_delim("f", delim=",")</code>	Any delimiter	<code>delim=","</code> or <code>delim="\t"</code>
<code>read_fwf("f", widths)</code>	Fixed-width	<code>fwf_widths(lengths, names)</code>
<code>read_excel("f.xlsx")</code>	Excel (readxl)	<code>sheet</code> , <code>range</code> , <code>col_names</code> , <code>skip</code>

col_types shortcuts & Tidy Data Rules

col_types: c=character, i=integer, d=double, n=number, l=logical, D=date, T=datetime, t=time, ?=guess, _=skip

`types = "ccccccciinnn" # 8 chars, 1 int, 3 numeric`

Tidy Data: (1) Each variable = one column. (2) Each observation = one row. (3) Each observational unit = one table.

Common Problems: column headers are values; multiple vars in one column; vars in both rows & columns; multiple units in same table.

SESSION 3: Data Structure — Long & Wide

pivot_longer() — Make Wide → Long

```

pivot_longer(data, cols, names_to="varname", values_to="valname", values_drop_na=TRUE,
names_transform=list(col=as.integer))

```

Use when: column headers are actually values (e.g. years, income groups)

pivot_wider() — Make Long → Wide

```

pivot_wider(data, names_from=col, values_from=col)

```

Use when: one column stores multiple variable names (e.g. TMAX/TMIN in "element" column)

Transposed Dataset (double pivot)

```

df %>% pivot_longer(-metric, names_to="planet") %>% pivot_wider(names_from=metric, values_from=value)

```

Type Conversion after Pivot

```

df %>% mutate(across(2:8, as.double)) %>% mutate(across(col, as.integer))

```

Note: pivot from mixed types → all become character. Use mutate/across to fix types after pivoting.

SESSION 4: Column & Row Operations

Key dplyr Verbs

Function	SQL Equivalent	Example
<code>select(df, col1, col2)</code>	SELECT	<code>select(df, -waste) # drop col</code>
<code>filter(df, condition)</code>	WHERE	<code>filter(df, age > 18 & !is.na(x))</code>
<code>arrange(df, col)</code>	ORDER BY	<code>arrange(df, desc(salary))</code>
<code>group_by(df, col)</code>	GROUP BY	<code>df %>% group_by(city)</code>
<code>summarise(df, n=n())</code>	aggregate	<code>summarise(df, mean_sal=mean(salary))</code>
<code>bind_rows(df1, df2)</code>	UNION ALL	same cols preferred
<code>bind_cols(df1, df2)</code>	JOIN (pos)	same number of rows

separate() — Split One Column into Many

```

separate(df, col, c("col1","col2"), sep="-") # by delimiter
separate(df, col, c("col1","col2"), sep=4) # by position
separate_wider_delim(df, cols=DRG, names=c(...), delim="-", too_many="merge")
separate_wider_position(df, cols=DRG, widths=c(A=3, 3, B=200), too_few="debug")

```

Warnings about "too many pieces" mean some rows have more delimiters. Use too_many="merge" to fix.

unite() — Merge Multiple Columns into One

```

unite(df, new_col, col1, col2, sep=" ", remove=FALSE)

```

remove=FALSE keeps original columns.

Pipe Operator & Workflow

```

inpatient %>% filter(!is.na(DRG)) %>% select(DRGcode, Name, City) %>% arrange(desc(Discharges))

```

Use |> (base R pipe, R 4.1+) or %>% (magrittr/tidyverse). Both work the same way.

SESSION 5: Missing Values & Duplicates

Special Numeric Values

Value	Meaning	Check with
NA	Missing value (like SQL NULL)	is.na(x)
NaN	Not a Number (0/0)	is.nan(x) — also returns T for is.na()
Inf / -Inf	Infinity (n/0)	is.infinite(x)
NULL	Empty object	is.null(x)

■ NaN is also NA. NA is NOT NaN. NaN and NA are neither finite nor infinite.

Filtering & Handling Missing Values

```
is.na(x) # check if NA df %>% filter(is.na(col)) # keep rows WITH missing df %>% filter(!is.na(col)) # keep rows WITHOUT missing
sum(x, na.rm=TRUE) # ignore NA in calculations mean(x, na.rm=TRUE) df$col[is.na(df$col)] <- mean(df$col, na.rm=TRUE) # impute with mean
```

Replace Special Values (ifelse approach)

```
df %>% mutate(col=ifelse(col=="", 0, col)) %>% mutate(col=ifelse(col=="Z", 0, col)) %>% mutate(col=as.numeric(col))
```

Strategy: (1) Detect. (2) Reconstruct from other sources/logic. (3) Interpolate/impute. (4) Ignore/remove.

Missing Data from Missing Rows (Hidden NAs)

```
# If entire observations are missing, add them manually: missing_rows <- tibble(State=c("Hawaii","Iowa"), value=c(0,0))
full_df <- bind_rows(original_df, missing_rows) # WARNING: mean(df) vs mean(full_df) can differ dramatically!
```

Duplicated Values

```
unique(df) # remove exact duplicate rows df[!duplicated(df), ] # keep first occurrence df %>% filter(!(City=="Tokyo" & Year==2020)) # remove specific rows
# For partial duplicates: decide which record is correct.
```

Units / Data Conversion

```
# Celsius/Fahrenheit conversion: weather %>% mutate(mintemp = mintemp/10) %>% # fix decimal issue mutate(maxtemp = maxtemp*(9/5)+32) # Celsius to Fahrenheit
```

Aggregate Functions on Data Frames

Function	Purpose
summary(df)	Min, Q1, Median, Mean, Q3, Max; counts NAs
head(df, n)	First n rows (default 6)
tail(df, n)	Last n rows (default 6)
glimpse(df)	Column types + preview (tidyverse)
nrow(df) / ncol(df)	Number of rows / columns
unique(df\$col)	Unique values in a column
table(df\$col)	Frequency table for one variable
prop.table(table(x))	Proportions; ,1 = row %, ,2 = col %

SESSION 6: Outliers

What are Outliers?

Data points distant from the norm. Can be errors OR true extreme values. Strategy:

(1) Detect → (2) Verify/Reconstruct → (3) Impute/Cap/Predict → (4) Remove value (→NA) → (5) Remove row

Detection Methods

Method	Code	Notes
Boxplot	<code>boxplot(df\$col)</code>	Visual; whiskers = $1.5 \times \text{IQR}$
Boxplot stats	<code>boxplot.stats(df\$col)\$out</code>	Returns outlier values (IQR method)
Z-score	<code>abs(scale(df\$col)) > 3</code>	Beyond 3 standard deviations
Manual filter	<code>df %>% filter(col > threshold)</code>	Based on domain knowledge
Quantile	<code>quantile(df\$col, c(0.05, 0.95))</code>	5th / 95th percentile thresholds
Summary	<code>summary(df\$col)</code>	Check Min and Max values
Bivariate	<code>boxplot(age ~ grade)</code>	Groupwise boxplot reveals cross-variable outliers

Handling Outliers

```
# Set to NA (unknown / likely erroneous): df <- df %>% mutate(Salary=ifelse(Salary>1000000, NA, Salary)) # Impute using domain model (e.g., age = grade + 5): df <- df %>% mutate(age=ifelse(studentID==10115, 7, age)) %>% mutate(age=ifelse(studentID==10116, 12, age)) # Cap (winsorize): replace with threshold df <- df %>% mutate(col=ifelse(col>cap, cap, col)) # Remove outlier row: df <- df %>% filter(Salary <= 200000)
```

Bivariate Check — Important!

A value can look OK univariately but be wrong in context. Always check cross-variable consistency.

```
boxplot(tests$age ~ tests$grade) # age should increase with grade residents %>% filter(ownsHome == rentsHome) # logical contradiction check
```

■ Tip: Check logical constraints between variables (e.g., own AND rent = impossible).

Descriptive Stats Quick Reference

Function	Output
<code>summary(df)</code>	Min, Q1, Median, Mean, Q3, Max + NA count
<code>table(df\$sex, df\$edu)</code>	Cross-tabulation / contingency table
<code>prop.table(table(x), 1)</code>	Row proportions (margin=1)
<code>prop.table(table(x), 2)</code>	Column proportions (margin=2)
<code>sd(x, na.rm=T)</code>	Standard deviation
<code>var(x, na.rm=T)</code>	Variance
<code>median(x, na.rm=T)</code>	Median
<code>quantile(x, 0.25)</code>	25th percentile (Q1)
<code>IQR(x)</code>	Interquartile range (Q3 - Q1)

SESSION 7: Dates & Times (lubridate)

Date/Time Classes in R

Class	Stores	Notes
Date	Days since 1970-01-01	No time component; as.numeric() gives day count
POSIXct	Seconds since epoch	Default datetime; compact; use for data frames
POSIXlt	List of date parts	Access with \$ (e.g., \$hour, \$min); less efficient
difftime	Time difference	In seconds

Epoch in R: 1970-01-01 | Microsoft: 1900-01-00 | Get current: today() | now()

Date/Time Construction Functions

Input Format	Function	Example
January 12, 2026	mdy()	mdy("01/12/2026")
2026-01-12	ymd()	ymd("2026-01-12")
12/01/2026	dmy()	dmy("12/01/26")
2026-01-12 10:34:21	ymd_hms()	ymd_hms("2026-01-12 10:30:26", tz="EST")
January 12, 2026 10:34	mdy_hm()	mdy_hm("June 23, 1912, 02:15 AM", tz="Europe/London")

Extraction Functions

year(x) month(x) day(x) hour(x) minute(x) second(x) wday(x) # day of week (1=Sun) wday(x, label=TRUE) # "Mon", "Tue"...
wday(x, label=TRUE, abbr=FALSE) # "Monday"... yday(x) # day of year semester(x) # 1 or 2 am(x) / pm(x) # TRUE/FALSE dst(x)
daylight saving time? leap_year(x) # is it a leap year?

Time Zones

grep("Tokyo", OlsonNames(), value=TRUE) # find timezone name with_tz(datetime, "Asia/Tokyo") # convert to timezone
tz(datetime) # get timezone

Time Differences: duration vs. period vs. interval

Type	Description	Examples	Note
duration	Fixed time, ignores calendar	years(1) = 365.25d, ddays(1), dhours(1)	No dmonths!
period	Calendar-aware	years(1), months(1), weeks(1), days(1)	Respects leap years
interval	Start + end point	interval(start, end)	Use for precise spans

Age calculation: alan <- mdy_hm("June 23, 1912, 02:15 AM", tz="Europe/London") alandiff <- interval(alan, now())
alandiff / years(1) # uses periods – correct for leap years alandiff / dyears(1) # uses duration – approx 365.25 days #
Rounding: floor_date(now(), unit="month") # round down to month start ceiling_date(now(), unit="minutes") # round up to
next minute round_date(now(), unit="hour") # round to nearest hour

SESSION 8: String Manipulation (stringr)

Core stringr Functions

Function	Purpose	Example
<code>str_length(x)</code>	Length of string	<code>str_length("Hello")</code> → 5
<code>str_c(..., sep)</code>	Concatenate strings	<code>str_c("A","B",sep="-")</code> → "A-B"
<code>str_sub(x, start, end)</code>	Extract substring by position	<code>str_sub("Bruce",-3,-1)</code> → "uce"
<code>str_to_upper(x)</code>	Convert to UPPERCASE	<code>str_to_upper("hello")</code> → "HELLO"
<code>str_to_lower(x)</code>	Convert to lowercase	
<code>str_trim(x, side)</code>	Remove whitespace	<code>side="both","left","right"</code>
<code>str_detect(x, pat)</code>	Does pattern exist? → logical	<code>str_detect(x,"pepper")</code> → T/F
<code>str_subset(x, pat)</code>	Keep strings matching pattern	Returns matching strings
<code>str_count(x, pat)</code>	Count matches in string	<code>str_count("pepper","p")</code> → 3
<code>str_extract(x, pat)</code>	Extract first match	<code>str_extract(text,"\d+")</code>
<code>str_extract_all(x, pat)</code>	Extract all matches → list	<code>str_extract_all(text,"\d+")</code>
<code>str_replace(x, pat, repl)</code>	Replace first match	<code>str_replace(x,"ID#:", "")</code>
<code>str_replace_all(x, pat, repl)</code>	Replace all matches	<code>str_replace_all(x,".",",")</code>
<code>str_split(x, pat)</code>	Split string → list	<code>str_split(x,fixed("-"))</code>

str_c vs paste

```
str_c("A", NA, sep=" ") → NA # str_c propagates NA paste("A", NA, sep=" ") → "A NA" # paste converts NA to text
```

Practical Example: Standardizing Data

```
# Uppercase everything, then fix bad values: df <- df %>% mutate(Region = str_to_upper(Region)) %>% mutate(Region = ifelse(Region %in% bad_vals, "CORRECT", Region)) %>% mutate(Region = ifelse(Region == "CHICAGO, NA", "CHICAGO, IL", Region)) %>% mutate(Region = ifelse(Region %in% nacodes, NA, Region)) # Find and replace alternate spellings: alternates <- df %>% filter(grepl("McDo", DBAName, ignore.case=TRUE)) %>% pull(DBAName) %>% unique() df <- df %>% mutate(DBAName = ifelse(DBAName %in% alternates, "MCDONALDS", DBAName))
```

Base R String Functions

```
nchar(x) # string length (not for factors) substr(x, start, stop) # extract by position grep(pattern, x) # returns indices of matches grepl(pattern, x) # returns logical vector sub(pattern, replacement, x) # replace FIRST match gsub(pattern, replacement, x) # replace ALL matches regexr(pattern, x) # position of FIRST match gregexpr(pattern, x) # positions of ALL matches → list regmatches(x, m) # extract matches from regexr/gregexpr
```

SESSION 8-9: Regular Expressions (Regex)

Meta Characters & Special Tokens

Pattern	Matches	Pattern	Matches
.	Any single character	^	Start of string
\$	End of string	\\w	Word char [a-zA-Z0-9_]
\\W	Non-word character	\\d	Digit [0-9]
\\D	Non-digit	\\s	Whitespace (space, tab)
\\S	Non-whitespace	\\t	Tab character
[abc]	a, b, or c	[a-z]	Any lowercase letter
[A-Z]	Any uppercase letter	[0-9]	Any digit
[^abc]	NOT a, b, or c		OR (alternation)
()	Grouping	\\	Escape next char

Quantifiers (refer to preceding char or group)

Pattern	Meaning	Example	Matches
?	0 or 1 times	"ac?b"	"ab", "acb"
*	0 or more times	"ac*b"	"ab", "acb", "accb"...
+	1 or more times	"ac+b"	"acb", "accb"... (not "ab")
{m}	Exactly m times	"ac{2}b"	"accb"
{m,n}	Between m and n times	"ac{2,3}b"	"accb", "acccb"
{m,}	At least m times	"ac{2,}b"	"accb", "acccb", "accccb"...

Common Regex Patterns

Pattern	Matches
"^[a-z]{3,5}\$"	Lowercase words 3-5 chars long
"^[+-]?([0-9][0-9]*)\$"	Signed integers (+1, -99, 0)
"\\d{4}"	Exactly 4 digits
"\\d{4}\$"	4 digits at end of string
"^[A-Z]{3}\$"	Exactly 3 uppercase letters
"\\w+@\\w+\\.\\w+"	Email address pattern
"https?://\\S+"	URL starting with http/https
"[A-Z][a-z]+"	Word starting with capital letter
"\\d{4}-\\d{2}-\\d{2}"	Date in YYYY-MM-DD format
"\\d{2}/\\d{2}/\\d{4}"	Date in DD/MM/YYYY format
"<.+>"	HTML tag (lazy match)
"<.+>"	HTML tags (greedy — matches too much)

Greedy vs. Lazy Matching

```
string <- "HTML tag example"
regexr("<.+>", string) # GREEDY: matches "<EM>first</EM>"
regexr("<.+?>", string) # LAZY: matches "<EM>" only
# Add ? after quantifier to make it lazy: *? +? {m,n}?
```

Sub-Pattern Capture Groups

```
# Extract coordinate components:
string <- c("(1,2)", "(-2, 7)", "(a, 3)")
pattern <- "\\(\\s*([+-]?([0-9][0-9]*))\\s*,\\s*([+-]?([0-9][0-9]*))\\s*\\)"
komp1 <- sub(pattern, "\\1", string) # group 1
komp2 <- sub(pattern, "\\3", string) # group 3 (nested!)
# Groups numbered left-to-right by opening parenthesis
```

regexr / gregexpr / regmatches

```
# Find FIRST match position:
first_space <- regexr("\\s", text1)
# Find ALL match positions:
string_pos <- gregexpr("\\d", text1)
# Extract matched text from gregexpr result:
regmatches(text, gregexpr("\\d+", text)) # → list of matches
# Extract substring using position:
substr(text2, string_pos4, string_pos4 + 1)
```

ADDITIONAL TOPICS & PRACTICAL NOTES

Joining Data Frames (dplyr)

Function	Keeps rows from...	SQL Equivalent
<code>inner_join(x, y, by="key")</code>	Both x AND y (matched only)	INNER JOIN
<code>left_join(x, y, by="key")</code>	All of x, matched from y	LEFT JOIN
<code>right_join(x, y, by="key")</code>	All of y, matched from x	RIGHT JOIN
<code>full_join(x, y, by="key")</code>	All of x AND all of y	FULL OUTER JOIN
<code>semi_join(x, y, by="key")</code>	Rows in x that match y (no y cols)	WHERE EXISTS
<code>anti_join(x, y, by="key")</code>	Rows in x that do NOT match y	WHERE NOT EXISTS

```
# Join on differently named keys: left_join(x, y, by=c("ID"="ProviderID")) # Join on multiple keys: inner_join(x, y, by=c("City", "Year"))
```

Other File Formats & Special Import Cases

Format	Package	Function	Notes
JSON	jsonlite	<code>fromJSON("file.json")</code>	Returns list or data frame
SPSS (.sav)	haven	<code>read_sav("file.sav")</code>	Preserves labels
Stata (.dta)	haven	<code>read_dta("file.dta")</code>	Preserves labels
SAS (.sas7bdat)	haven	<code>read_sas("file.sas")</code>	Preserves formats
R object (.rds)	base	<code>readRDS("file.rds")</code>	Single R object
R workspace (.RData)	base	<code>load("file.RData")</code>	Multiple objects
Web page (HTML)	rvest	<code>read_html(url)</code>	Then <code>html_table()</code> / <code>html_text()</code>
Fixed-width (.txt)	readr	<code>read_fw(f, fwf_widths(...))</code>	Need column widths

```
# Web scraping example: library(rvest) html <- read_html("https://en.wikipedia.org/wiki/...") table_data <- html_table(html_element(html, ".wikitable")) # Use CSS selectors or XPath to target elements
```

`mutate()`, `ifelse()` & `case_when()` — Column Transformations

```
# ifelse: binary condition df %>% mutate(category = ifelse(score >= 60, "Pass", "Fail")) # case_when: multiple conditions (like SQL CASE WHEN) df %>% mutate(grade = case_when( score >= 90 ~ "A", score >= 75 ~ "B", score >= 60 ~ "C", TRUE ~ "F" # TRUE = else / default )) # across: apply same function to many columns at once df %>% mutate(across(c(col1, col2, col3), as.numeric)) df %>% mutate(across(where(is.character), str_to_upper))
```

`group_by` + summarise Patterns

```
df %>% group_by(City, Year) %>% summarise( n = n(), avg_charge = mean(AverageCharges, na.rm=TRUE), max_charge = max(AverageCharges, na.rm=TRUE), n_missing = sum(is.na(AverageCharges)) ) %>% arrange(desc(avg_charge))
```

Critical Rules & Common Exam Traps

Topic	Rule / Trap
Regex in R strings	"\\d" in string = \d in regex (each \ must be doubled)
is.na() trap	<code>is.na(A B)</code> is WRONG. Must write: <code>is.na(A) is.na(B)</code>
NA propagation	<code>5 + NA = NA</code> . Always add <code>na.rm=TRUE</code> to mean/sum/max/min
NaN vs NA	<code>is.na(NaN) = TRUE</code> , but <code>is.nan(NA) = FALSE</code> . Neither is finite.
Pivot types	<code>pivot_longer</code> on mixed types makes everything character. Fix with <code>across()</code> .
separate() warning	"Too many pieces" = multiple delimiters. Use <code>too_many="merge"</code>
str_c vs paste	<code>str_c("A", NA) = NA</code> . <code>paste("A", NA) = "A NA"</code> . <code>str_c</code> propagates NA.
dmonths()	Does NOT exist in lubridate! Use <code>months()</code> (period) instead.
period vs duration	<code>dyears(1) = 365.25 days</code> (ignores calendar). <code>years(1)</code> respects leap years.
Greedy regex	"<.+>" matches too much. Use "<.+?>" (lazy) to match minimum.
Boxplot IQR rule	Outlier = beyond $Q1 - 1.5 * IQR$ or $Q3 + 1.5 * IQR$. NOT beyond $mean + 3 * SD$.
Hidden NAs	Entire rows can be missing from a dataset. <code>bind_rows()</code> to add them back.
filter() logic	<code>filter()</code> KEEPS rows where condition is TRUE. Use <code>!</code> to negate.
Epoch difference	R epoch = 1970-01-01. Excel/Microsoft epoch = 1900-01-00.
library(readxl)	<code>readxl</code> installs with tidyverse but must be loaded SEPARATELY.